Generation of DTMF signals


```
clear all;
close all;
t = -2: 0.05 : 2;
x = input ('enter the input
          number');
fr1 = 697;
fr2 = 770;
fr3 = 852;
fr4 = 941;
fc1 = 1209;
fc2 = 1336;
fc3 = 1477;
fc4 = 1633;
Y0 = sin (2*pi*fr4*t) +
     sin(2*pi*fc2 * t) ;
Y1 = sin(2 * pi * fr1 * t) +
     sin(2 * pi * fc1 * t);
Y2 = sin(2* pi * fr1 * t) +
     sin(2*pi * fc2 * t);
Y3 = sin(2* pi * fr1 * t) +
     sin(2*pi*fc3*t);
Y4 = sin(2* pi * fr2 * t) +
     sin(2* pi * fc1 * t);
Y5 = sin(2* pi * fr2 * t) +
     sin(2 * pi * fc2 * t);
Y6 = sin (2* pi * fr2 * t) +
     sin(2 * pi * fc3 * t);
Y7 = sin(2*pi*fr3*t) +
     sin(2 * pi * fc1 * t);
Y8 = sin(2* pi * fr3 * t) +
     sin(2* pi * fc2 * t);
Y9 = sin (2* pi * fr3 * t) +
     sin (2* pi * fc3 * t);
Y_start = sin (2* pi* fr4 *t) +
          sin (2* pi * fc1 * t);
Y_canc = sin (2* pi * fr4 * t) +
         sin (2* pi * fc3 * t);
if (x==1)
 plot (t, Y1)
 xlabel ('time (t)')
 ylabel ('amplitude')
else if (x==2)
 plot (t, Y2)
 xlabel ('time (t)')
 ylabel ('amplitude')
else if (x==3)
 plot (t, Y3)
 xlabel ('time (t)');
 ylabel ('amplitude')
else if (x==4)
 plot (t, Y4)
xlabel ('time (t)')
Ylabel ('amplitude')
```

cont ...

DSP = D , ~ 0.

and sin = 0.05

Generation of DTMF signals


```
clear all;
close all;
t = -2: 0.05 : 2;
x = input ('enter the input number');
fr1 = 697;
fr2 = 770;
fr3 = 852;
fr4 = 941;
fc1 = 1209;
fc2 = 1336;
fc3 = 1477;
fc4 = 1633;
y0 = sin (2*pi*fr4*t) +
     sin(2*pi*fc2*t);
y1 = sin(2*pi*fr1*t) +
     sin(2*pi*fc1*t);
y2 = sin(2*pi*fr1*t) +
     sin(2*pi*fc2*t);
y3 = sin(2*pi*fr1*t) +
     sin(2*pi*fc3*t);
y4 = sin(2*pi*fr2*t) +
     sin(2*pi*fc1*t);
y5 = sin(2*pi*fr2*t) +
     sin(2*pi*fc2*t);
y6 = sin(2*pi*fr2*t) +
     sin(2*pi*fc3*t);
y7 = sin(2*pi*fr3*t) +
     sin(2*pi*fc1*t);
y8 = sin(2*pi*fr3*t) +
     sin(2*pi*fc2*t);
y9 = sin(2*pi*fr3*t) +
     sin(2*pi*fc3*t);
y_start = sin(2*pi*fr4*t) +
          sin(2*pi*fc1*t);
y_canc = sin(2*pi*fr4*t) +
         sin(2*pi*fc3*t);
if (x==1)
 plot(t, y1)
 xlabel ('time(t)')
 ylabel ('amplitude')
else if (x==2)
 plot(t, y2)
 xlabel ('time(t)')
 ylabel ('amplitude')
else if (x==3)
 plot(t, y3)
 xlabel ('time(t)');
 ylabel ('amplitude')
else if (x==4)
 plot(t, y4)
xlabel ('time(t)')
ylabel ('amplitude')
```

cont ...

DSP = D , ~ 0 ... from 2nd sin = 0.05

Implementation of LP IIR Filter

```
clc;
clear all;
close all;
disp('enter the IIR filter design specifications');
rp = input ('enter the passband ripple:');
rs = input ('enter the stopband ripple:');
wp = input ('enter the passband freq:');
ws = input ('enter the stopband freq:');
fs = input ('enter the sampling freq:');
w1 = 2*wp/fs; w2 = 2*ws/fs;
[n, wn] = buttord (w1, w2, rp, rs, 's');
disp ('Freq response of IIR LPF is:');
[b, a] = butter (n, wn, 'low', 's');
w = 0:.01:pi;
[h, om] = freqs (b, a, w);
m = 20*log10(abs(h));          --> magnitude.
an = angle(h);
figure, subplot (2,1,1); plot (om/pi, m);
title ('magnitude response of IIR filter is:');
xlabel ('(a) Normalized freq.-->');
```

Implementation of LP IIR Filter

```
clc;
clear all;
close all;
disp('enter the IIR filter design specifications');
rp = input ('enter the passband ripple:');
rs = input ('enter the stop band ripple:');
wp = input ('enter the passband freq:');
ws = input ('enter the stopband freq:');
fs = input ('enter the sampling freq:');
w1 = 2*wp/fs; w2 = 2*ws/fs;
[n, wn] = buttord (w1, w2, rp, rs, 's');
disp ('Freq response of IIR LPF is:');
[b, a] = butter (n, wn, 'low', 's');
w = 0:.01:pi;
[h, om] = freqs (b, a, w);
m = 20* log10(abs(h));          → magnitude.
an = angle(h);
figure, subplot (2,1,1); plot (om/pi, m);
title ('magnitude response of IIR filter is:');
xlabel ('(a) Normalized freq.-->');
```

```
ylabel ('Gain in dB ---->');
subplot (2,1,2); plot (om/pi, an);
title ('phase response of IIR filter is:');
xlabel ('(b) Normalized freq. ---->');
ylabel ('phase in radians ---->');
```

W_1 = normalized pass band freq

W_2 = " stop band "

buttord = butterworth filter order.

Butterworth filter = butter (n, w_n)

Implementation of HP IIR Filter

```
clc;
clear all;
close all;
disp ('enter the IIR filter design specifications');
rp = input ('enter the passband ripple');
rs = input ('enter the stopband ripple');
wp = input ('enter the passband freq');
ws = input ('enter the stopband freq');
fs = input ('enter the sampling freq');
w1 = 2* wp/fs ; w2 = 2*ws/fs;
[n, wn] = buttord (w1, w2, rp, rs, 's');
disp ('Freq response of IIR HPF is:');
[b,a] = butter (n, wn, 'high', 's');
w = 0:.01: pi;
[h,om] = freqs (b,a,w);
m = 20 * log 10 (abs(h));
an = angle(h);
figure, subplot (2,1,1);
  plot (om/pi, m);
title ('magnitude response of IIR filter is:')
xlabel ('(a) Normalized freq. -->');
ylabel ('Gain in dB -->');
```

```
subplot (2,1,2);
plot (om/pi, an);
title ('phase response of IIR filter is :');
xlabel ('b) Normalized freq. ->');
ylabel ('phase in radians ->');
```

Generation of sinusoidal signal through filtering

```
close all;
clear all;
b = [1];
a = [1, -1, 0.9];
n = [-20 : 120];
t = 0 : 0.1 : 2 * pi;
x = sin(t)
s = filter(b, a, x);
stem(t, s);
title('sinusoidal response');
xlabel('n');
ylabel('s(n)');
```

```
else if (x==5)
plot (t, y5)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==6)
plot (t, y6)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==7)
plot (t, y7)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==8)
plot (t, y8)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==9)
plot (t, y9)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==0)
plot (t, y0)
xlabel ('time(t)')
ylabel ('amplitude')
```

```
elseif (x==11)
plot (t, y_start)
xlabel ('time(t)')
ylabel ('amplitude')
else if (x==12)
plot (t, y_conc)
xlabel ('time(t)')
ylabel ('amplitude')
else
disp('enter the correct
                input')
end
```

```
);
idal);
esoidal);
```

Implementation of Decimation process

```
clc;
clear all;
close all;
D = input ('enter the downsampling factor');
L = input ('enter the length of the input signal');
f1 = input ('enter the frequency of first sinusoidal');
f2 = input ('enter the frequency of second sinusoidal');
n = 0:L-1;
x = sin(2*pi*f1*n) + sin(2*pi*f2*n);
y = decimate (x, D, 'fir');
subplot (2,1,1);
stem (n, x(1:L));
title ('input sequence');
xlabel ('time(n)');
ylabel ('amplitude');
subplot (2,1,2)
m = 0:(L/D)-1;
stem (m, y(1:L/D));
title ('Decimated sequence');
xlabel ('time(n)');
ylabel ('amplitude');
```

DSP = 5 , Length = 100 , freq 1st sin = 0.01
2nd sin = 0.03

Implementation of interpolation process

```
clc;
clear all;
close all;
L = input ('enter the upsampling factor');
N = input ('enter the length of the input signal');
f1 = input ('enter the frequency of first sinusoidal');
f2 = input ('enter the frequency of second sinusoidal');
n = 0: N-1;
x = sin (2 * pi * f1 * n) + sin (2 * pi * f2 * n);
y = interp (x, L);
subplot (2,1,1)
stem (n, x(1: N))
title ('input sequence');
xlabel ('time(n)');
ylabel ('amplitude');
subplot (2, 1, 2)
stem (m, y (1: N * L))
title ('output sequence');
xlabel ('time(n)');
ylabel ('amplitude');
```

m = 0 : N * L - 1;

~~stem (m, x(1: N))~~
~~title ('input sequence');~~
~~xlab~~

up s.f. = 5
length = 9
freq. 1st sin = 0.1
freq 2nd sin = 0.3

Implementation of I/D sampling rate converters

```
clc;
clear all;
close all;
L = input ('enter the upsampling factor');
D = input ('enter the down sampling factor');
N = input ('enter the length of the i/p signal');
                                        input
f1 = input ('enter the frequency of first sinusoidal');
f2 = input ('enter the frequency of second sinusoidal')
n = 0: N-1
x = sin (2*pi * f1 * n) + sin (2*pi * f2 *n);
Y = resample (x, L, D);
subplot (2,1,1)
stem (n, x(1:N))
title ('input sequence');
xlabel ('time (n)');
ylabel ('amplitude');
subplot (2,1,2)
m = 0: N*L/D-1;
stem (m, Y(1:N*L/D));
title ('output sequence');
xlabel ('time (n)');
ylabel ('amplitude');
```

upsamp f. = 3
down " " = 2
length = 40
freq. 1st sin = 0.01
freq 2nd sin = 0.03

Impulse response of 1st & 2nd order systems

```
clc;
close . all;
clear all;
n = 0:10;
b = [2 0 0];
a = [1 -0.9 0];
y = dimpulse (b, a, length (n));
subplot (2,1,1);
stem (n, y);
xlabel ('n-->');
ylabel ('amplitude');
title ('impulse response of first order system');
b = [1 0 0];
a = [1 0.6 0.8];
y1 = dimpulse (b, a, length (n));
subplot (2,1,2);
stem (n, y1);
xlabel ('n-->');
ylabel ('amplitude');
title ('impulse response of second order system');
```